
CASE STUDY REPORT

**Software Development Company: Git- Based Collaborative
Software Development Workflow**

PREET HANSALIYA [92510103060]
INDRAPRIT JADAV [92510103055]
OM BERA [92510103074]

DEPARTMENT
COMPUTER ENGINEERING

ACADEMIC YEAR
2026-2027

SEMESTER
5TH

SUBJECT
OPEN SOURCE TECHNOLOGIES (01CE0526)

CLASS
5EC6

BATCH
A-B

CASE STUDY REPORT

RUBRICS FOR CASE STUDY

Sr. No.	Assessment Criteria	Excellent	Good	Satisfactory	Poor	Marks
1	Problem Analysis, Cloud Concepts & SDG Mapping	4 Marks: Clear problem identification, appropriate cloud concepts, and accurate SDG mapping.	3 Marks: Good analysis with minor gaps in concepts or SDG mapping.	1-2 Marks: Basic problem understanding with partial cloud concepts or SDG mapping.	0 Marks: Inadequate or incorrect analysis and no SDG mapping.	4
2	Security Analysis & Technical Depth	4 Marks: Comprehensive security analysis, well-defined architecture, appropriate technologies, and justified solutions.	3 Marks: Good technical explanation with minor omissions.	1-2 Marks: Limited security analysis and technical details.	0 Marks: No meaningful security analysis or technical explanation.	4
3	Report Quality & References	4 Marks: Well-structured report in IEEE format with comprehensive technical content, proper citations, and relevant references.	3 Marks: Good report structure with minor formatting or referencing issues.	1-2 Marks: Basic report with limited technical details or incomplete references.	0 Marks: Incomplete report with poor documentation and missing references.	4
	Total					12

Sr. No.	Case Study Title / Problem Statement	COs	Date	R1(4)	R2(4)	R3(4)	Marks (12)	Signature
1	Software Development Company: Git- Based Collaborative Software Development Workflow							

Ms. Priyanka Mangi
Subject Faculty

CASE STUDY REPORT

Software Development Company: Git- Based Collaborative Software Development Workflow
Course Name and Course Name: OPEN SOURCE TECHNOLOGIES (01CE0526)

Co's Mapped - CO2

SDG's Mapped - SDG 9, SDG 17

Abstract

Keywords

I. Introduction

- Background of the selected case study
- Importance of Open Source Technologies in the selected domain
- Objectives of the case study
- Scope of the study

II. Organization/Application Background

- Overview of the organization/application/system
- Existing software ecosystem
- Business requirements
- Need for adopting Open Source Technologies

III. Problem Analysis

- Problem identification
- Existing challenges
- Engineering requirements
- Functional and non-functional requirements
- Constraints and assumptions

IV. Open Source Solution Analysis

- Selected Open Source technologies/frameworks
- Justification for technology selection
- Open Source license used (GPL, MIT, Apache, BSD, etc.)
- Comparison with proprietary alternatives
- Benefits of adopting OSS

V. System Architecture and Technology Stack

- Overall architecture diagram
- Technology stack
- Version Control System (Git/GitHub/GitLab)
- Web framework (Laravel/Django/Node.js)
- Database (MySQL/PostgreSQL/MongoDB)
- Web Server (Apache/Nginx)
- Development and deployment workflow

VI. Database and Deployment Analysis

- Database design
- CRUD operations
- Database architecture
- Deployment strategy
- Reverse Proxy / Load Balancing (if applicable)

CASE STUDY REPORT

VII. Software Quality and Testing

- Testing strategy
- Unit Testing (PyTest/PHPUnit/JUnit)
- API Testing (Postman)
- Web Testing (Selenium)
- Load Testing (JMeter)
- Security Testing (OWASP ZAP)
- Testing outcomes and observations

VIII. Open Source Community and Licensing Analysis

- Open Source licensing model
- Compliance with license terms
- Community support
- Documentation quality
- Version management
- Contribution opportunities

IX. Technical Analysis

- Working of the proposed solution
- Advantages
- Limitations
- Scalability
- Reliability
- Maintainability
- Performance analysis

X. SDG Mapping and Justification

- Identify the relevant Sustainable Development Goal(s)
- Explain how the proposed Open Source solution contributes towards achieving the selected SDGs
- Justification with real-world impact

XI. Challenges and Recommendations

- Technical challenges
- Security challenges
- Deployment challenges
- Future improvements
- Recommendations for better implementation

XII. Conclusion

- Summary of the case study
- Key technical findings
- Learning outcomes
- Industrial relevance of Open Source Technologies

References

- IEEE Reference Format
- Minimum 5–8 authentic references
- Research papers

CASE STUDY REPORT

- Official Documentation
- GitHub Documentation
- Linux Foundation Documentation
- Books

Appendix (Optional)

- System Architecture Diagram
- Database Schema
- Git Workflow Diagram
- Screenshots
- Test Reports
- Deployment Screenshots
- Configuration Files
- Additional Tables

CASE STUDY REPORT

Software Development Company: Git- Based Collaborative Software Development Workflow

Preet Hansaliya
Department of Computer Engineering
Marwadi University
Rajkot, India
preet.hansaliya142520@marwadiuniversity.ac.in

Indraprit Jadav
Department of Computer Engineering
Marwadi University
Rajkot, India
IndiaIndraprit.jadav142533@marwadiuniversity.ac.in

Om Bera
Department of Computer Engineering
Marwadi University
Rajkot, India
beraom.143223@marwadiuniversity.ac.in

Abstract— This case study presents a Git-based collaborative software development workflow for a five-member team developing a Food Delivery Application. The existing development process suffers from overwritten changes, inconsistent code versions, frequent merge conflicts, and delays caused by simultaneous modification of common files. A structured workflow using Git and GitHub is proposed to isolate development through feature branches and integrate changes through Pull Requests and code review. The case study defines a branching strategy, branch naming convention, repository workflow, Pull Request process, branch protection rules, and a practical merge-conflict resolution procedure. A representative conflict scenario is also described to demonstrate how Git identifies competing changes and how developers can safely resolve them. Testing, security, maintainability, scalability, and Sustainable Development Goal mapping are included. The proposed workflow is intended to improve collaboration and reduce uncontrolled changes while keeping the main branch stable. Since the case study does not include evidence from an actual deployed project, implementation outcomes are presented as proposed or illustrative results rather than measured experimental results.

Keywords— *Git, GitHub, Version Control, Feature Branch, Pull Request, Merge Conflict, Code Review, Collaborative Software Development.*

I. INTRODUCTION

A. Background of the Case Study

Modern software applications are commonly developed by multiple programmers working on different modules at the same time. Without version control and a defined

collaboration process, developers may overwrite changes, create incompatible versions, or face difficulties when combining their work.

Git is a distributed version control system that records changes and allows developers to work independently through branches. GitHub provides repository hosting and collaboration features such as Pull Requests, code reviews, and branch protection. [1], [2]

In this case study, Git and GitHub are applied to a five-developer Food Delivery Application. The proposed workflow separates feature development from the stable main branch and provides a controlled method for integrating completed work.

B. Purpose of the Case Study

The purpose of this case study is to design a practical Git-based collaboration workflow that reduces conflicts and prevents uncontrolled changes to the main codebase.

The major objectives are:

- To design a suitable branching strategy.
- To define feature-branch development.
- To establish a Pull Request and code-review workflow.
- To demonstrate merge-conflict identification and resolution.
- To define basic testing and repository-security practices.

C. Scope

The study covers Git repository management, branch creation, feature development, commits, remote synchronization, Pull Requests, code review, branch protection, merge-conflict resolution, and final integration.

CASE STUDY REPORT

The application itself is considered a representative software project. Actual application performance, deployment results, and measured development improvements are outside the scope of this case study.

II. CASE STUDY BACKGROUND

A. Software Development Company

The case considers a hypothetical software development company working on web and mobile-based applications. The development team uses Git for local version control and GitHub as the central remote repository.

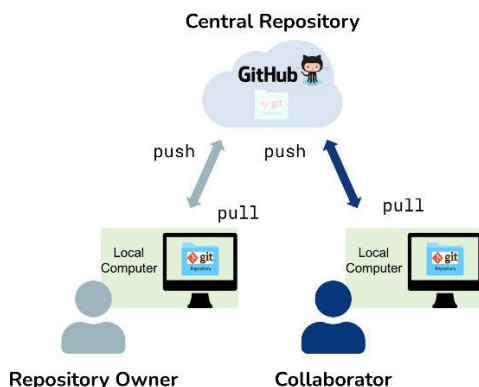
The company requires developers to work simultaneously without directly modifying the stable codebase. Therefore, a controlled branching and review process is required.

I. B. Food Delivery Application

The selected application is a Food Delivery Application containing common modules such as:

- User registration and authentication.
- Restaurant and menu management.
- Food ordering.
- Cart management.
- Payment processing.
- Order tracking.
- Delivery management.
- Administrative functions.

Several developers may need to modify common files such as routing, order services, authentication modules, and database models. This creates a possibility of conflicting changes.



[FIGURE 1: Git-Based Collaborative Development Workflow]

II. C. Development Team

The team consists of five developers. Responsibilities are divided according to application modules while following the same Git workflow.

TABLE I. DEVELOPER RESPONSIBILITIES

Developer	Primary Responsibility	Example Branch
Developer 1	Authentication and users	feature/user-auth
Developer 2	Restaurant and menu	feature/restaurant-menu
Developer 3	Cart and orders	feature/order-cart
Developer 4	Payment module	feature/payment
Developer 5	Delivery and tracking	feature/delivery-tracking

Developers are allowed to work independently on feature branches. Changes are integrated into main only through the defined review process.

III. PROBLEM STATEMENT

A. Existing Problem

The original development situation assumes that developers frequently modify common project files without a controlled branching process. One developer may save changes while another developer is working on an earlier version of the same file.

As a result, changes may be overwritten or Git may be unable to automatically combine different modifications. This produces merge conflicts and increases integration effort.

B. Impact of the Problem

The major effects include:

- Loss or overwriting of code changes.
- Merge conflicts during integration.
- Inconsistent versions between developers.
- Difficulty identifying the source of changes.
- Delayed feature integration.
- Increased possibility of defective code reaching the main branch.

Frequent integration and controlled source-code management can reduce the amount of work accumulated before integration.

CASE STUDY REPORT

C. Engineering Requirements

The proposed workflow should provide:

1. A stable central branch.
2. Isolated feature development.
3. Meaningful and traceable commits.
4. Regular synchronization with the remote repository.
5. Pull Request-based integration.
6. Code review before merging.
7. Automated checks where applicable.
8. A defined conflict-resolution procedure.
9. Restricted direct modification of main.
10. Clear responsibility for each feature.

IV. PROBLEM ANALYSIS

A. Root Causes

The main causes of the identified problem are:

- Direct development on the main branch.
- Multiple developers modifying the same files.
- Lack of branch naming standards.
- Infrequent synchronization with main.
- Large or unclear commits.
- Absence of mandatory code review.
- No defined conflict-resolution procedure.

The problem is therefore not caused by Git itself. It results mainly from uncontrolled collaboration practices.

B. Engineering Challenges

The team must balance independent development with regular integration. A branch should isolate incomplete work without remaining disconnected from the latest project state for too long.

Another challenge is deciding which change should be retained when two developers modify the same section of a file. Such conflicts require understanding the intended application behavior rather than simply selecting one version.

C. Complexity of the Problem

The application contains interconnected modules. A change in one component may affect another component. For example, a change in the order model may affect payment and delivery services.

Therefore, successful integration requires both version-control knowledge and understanding of the application.

D. Design Constraints

The workflow is designed under the following assumptions:

- Five developers use the same GitHub repository.
- main represents the stable codebase.
- Each feature is developed in a separate branch.
- Developers have suitable GitHub repository permissions.
- The application uses a shared source-code repository.
- Actual production deployment is not considered.
- Results are proposed unless supported by implementation evidence.

V. OPEN-SOURCE TECHNOLOGY SELECTION

A. Git

Git is a distributed version control system designed to track changes and support branching, merging, and collaborative development. Each developer can maintain a local repository while synchronizing changes with the shared remote repository.

Git provides the core operations required for this case study, including clone, switch, pull, add, commit, push, and merge.

B. GitHub

GitHub provides remote repository hosting and collaboration facilities around Git. Its Pull Request system allows developers to propose changes, discuss modifications, perform reviews, and merge approved work.

GitHub also supports protected branches with requirements such as reviews and status checks.

C. Technology Selection Justification

Git and GitHub are suitable because they directly address the identified collaboration problems. Git provides distributed version control and branching, while GitHub provides a shared repository and structured review process.

The combination separates development from integration and provides a traceable history of changes.

D. Advantages and Limitations

TABLE II. TECHNOLOGY ADVANTAGES AND LIMITATIONS

Technology	Advantages	Limitations
Git	Branching, history, merging, local version control	Requires Git knowledge

CASE STUDY REPORT

GitHub	Pull Requests, reviews, repository collaboration	Requires network access for remote collaboration
Git + GitHub	Controlled collaborative workflow	Poor practices can still cause conflicts

Git provides flexibility, but the team must establish rules for using that flexibility consistently.

VI. PROPOSED BRANCHING STRATEGY

A. Main Branch

The main branch represents the stable version of the Food Delivery Application. Developers should not normally push unfinished features directly to this branch.

Completed changes are integrated through Pull Requests after review and required checks.

B. Feature Branches

Each feature is developed in an independent branch created from the latest main branch.

Example:

```
git switch main
```

```
git pull origin main
```

```
git switch -c feature/payment
```

Feature branches isolate development and allow several developers to work simultaneously without directly affecting main.

C. Branch Naming Convention

A consistent naming convention makes the repository easier to understand.

TABLE III. BRANCH NAMING CONVENTION

Purpose	Format	Example
New feature	feature/<name>	feature/payment
Bug fix	fix/<name>	fix/order-total
Documentation	docs/<name>	docs/api-guide
Refactoring	refactor/<name>	refactor/order-service

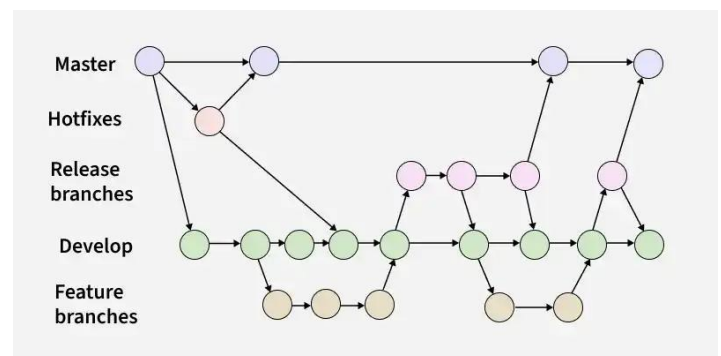
Branch names should be short, descriptive, and related to the actual work.

D. Branch Protection

The main branch should be protected so that developers cannot freely push changes to it. Suitable rules include:

- Pull Request required.
- At least one review required.
- Required status checks where automated testing exists.
- Direct force-push disabled.
- Branch deletion restricted.

GitHub supports these protections through branch protection rules.



[FIGURE 2: Proposed Branching Strategy]

VII. GIT COLLABORATIVE WORKFLOW

A. Repository Creation

A GitHub repository is created for the Food Delivery Application. The initial project is committed to the repository and the main branch is established.

A developer can obtain the project using:

```
"git clone https://github.com/company/food-delivery-app.git
cd food-delivery-app
```

The repository acts as the shared source-code location.

B. Updating the Main Branch

Before starting new work, developers should obtain the latest version of main.

```
git switch main
git pull origin main
```

CASE STUDY REPORT

This reduces the possibility of starting a feature from an outdated project state.

C. Creating a Feature Branch

The developer creates a branch for the assigned task.

`git switch -c feature/order-cart`

The developer then works only on the relevant feature instead of directly changing main.

D. Development and Commit

After completing a logical part of the feature, the developer checks the changes and creates a meaningful commit.

`git status`

`git add .`

`git commit -m "Add cart item quantity update"`

Commits should represent understandable units of work rather than unrelated collections of modifications.

E. Push to Remote Repository

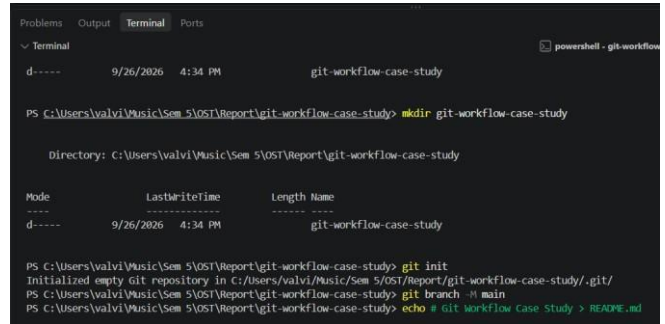
The feature branch is uploaded to GitHub:

`git push -u origin feature/order-cart`

The developer can then create a Pull Request from the feature branch to main.

TABLE IV. GIT WORKFLOW STEPS

Step	Activity	Main Command
1	Get repository	<code>git clone</code>
2	Update main	<code>git pull</code>
3	Create branch	<code>git switch -c</code>
4	Record changes	<code>git add, git commit</code>
5	Upload branch	<code>git push</code>
6	Review	Pull Request
7	Integrate	Merge



```

d----- 9/26/2026 4:34 PM git-workflow-case-study

PS C:\Users\valvi\Music\Sem 5\OST\Report\git-workflow-case-study> mkdir git-workflow-case-study

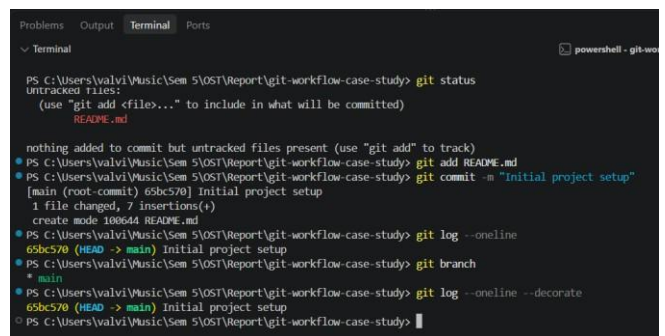
Directory: C:\Users\valvi\Music\Sem 5\OST\Report\git-workflow-case-study

Mode                LastWriteTime         Length Name
----                -
d----- 9/26/2026 4:34 PM git-workflow-case-study

PS C:\Users\valvi\Music\Sem 5\OST\Report\git-workflow-case-study> git init
Initialized empty Git repository in C:\Users\valvi\Music\Sem 5\OST\Report\git-workflow-case-study\.git\
PS C:\Users\valvi\Music\Sem 5\OST\Report\git-workflow-case-study> git branch -M main
PS C:\Users\valvi\Music\Sem 5\OST\Report\git-workflow-case-study> echo # git workflow Case Study > README.md

```

[SCREENSHOT 1: GitHub Repository]



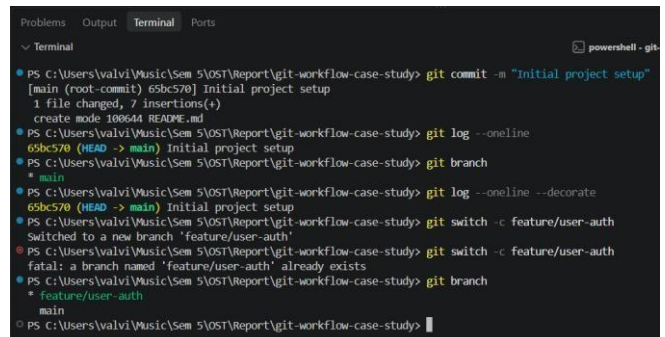
```

PS C:\Users\valvi\Music\Sem 5\OST\Report\git-workflow-case-study> git status
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        README.md

nothing added to commit but untracked files present (use "git add" to track)
PS C:\Users\valvi\Music\Sem 5\OST\Report\git-workflow-case-study> git add README.md
PS C:\Users\valvi\Music\Sem 5\OST\Report\git-workflow-case-study> git commit -m "Initial project setup"
[main (root-commit) 65bc570] Initial project setup
 1 file changed, 7 insertions(+)
 create mode 100644 README.md
PS C:\Users\valvi\Music\Sem 5\OST\Report\git-workflow-case-study> git log --oneline
65bc570 (HEAD -> main) Initial project setup
PS C:\Users\valvi\Music\Sem 5\OST\Report\git-workflow-case-study> git branch
* main
PS C:\Users\valvi\Music\Sem 5\OST\Report\git-workflow-case-study> git log --oneline --decorate
65bc570 (HEAD -> main) Initial project setup
PS C:\Users\valvi\Music\Sem 5\OST\Report\git-workflow-case-study>

```

[SCREENSHOT 2: Main Branch]

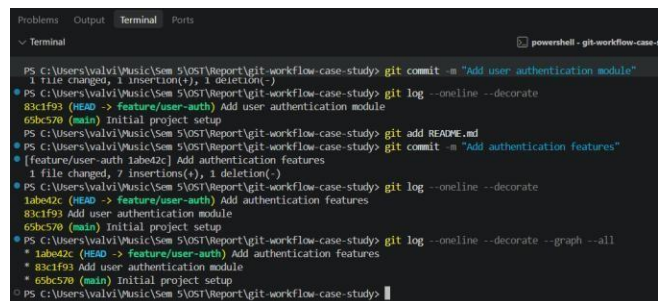


```

PS C:\Users\valvi\Music\Sem 5\OST\Report\git-workflow-case-study> git commit -m "Initial project setup"
[main (root-commit) 65bc570] Initial project setup
 1 file changed, 7 insertions(+)
 create mode 100644 README.md
PS C:\Users\valvi\Music\Sem 5\OST\Report\git-workflow-case-study> git log --oneline
65bc570 (HEAD -> main) Initial project setup
PS C:\Users\valvi\Music\Sem 5\OST\Report\git-workflow-case-study> git branch
* main
PS C:\Users\valvi\Music\Sem 5\OST\Report\git-workflow-case-study> git log --oneline --decorate
65bc570 (HEAD -> main) Initial project setup
PS C:\Users\valvi\Music\Sem 5\OST\Report\git-workflow-case-study> git switch -c feature/user-auth
Switched to a new branch 'feature/user-auth'
PS C:\Users\valvi\Music\Sem 5\OST\Report\git-workflow-case-study> git switch -c feature/user-auth
fatal: a branch named 'feature/user-auth' already exists
PS C:\Users\valvi\Music\Sem 5\OST\Report\git-workflow-case-study> git branch
* feature/user-auth
  main
PS C:\Users\valvi\Music\Sem 5\OST\Report\git-workflow-case-study>

```

[SCREENSHOT 3: Feature Branch]



```

PS C:\Users\valvi\Music\Sem 5\OST\Report\git-workflow-case-study> git commit -m "Add user authentication module"
1 file changed, 1 insertion(+), 1 deletion(-)
PS C:\Users\valvi\Music\Sem 5\OST\Report\git-workflow-case-study> git log --oneline --decorate
83c1f93 (HEAD -> feature/user-auth) Add user authentication module
65bc570 (main) Initial project setup
PS C:\Users\valvi\Music\Sem 5\OST\Report\git-workflow-case-study> git add README.md
PS C:\Users\valvi\Music\Sem 5\OST\Report\git-workflow-case-study> git commit -m "Add authentication features"
[feature/user-auth 1ab642c] Add authentication features
 1 file changed, 7 insertions(+), 1 deletion(-)
PS C:\Users\valvi\Music\Sem 5\OST\Report\git-workflow-case-study> git log --oneline --decorate
1ab642c (HEAD -> feature/user-auth) Add authentication features
83c1f93 Add user authentication module
65bc570 (main) Initial project setup
PS C:\Users\valvi\Music\Sem 5\OST\Report\git-workflow-case-study> git log --oneline --decorate --graph --all
* 1ab642c (HEAD -> feature/user-auth) Add authentication features
* 83c1f93 Add user authentication module
* 65bc570 (main) Initial project setup
PS C:\Users\valvi\Music\Sem 5\OST\Report\git-workflow-case-study>

```

[SCREENSHOT 4: Commit History]

CASE STUDY REPORT

VIII. PULL REQUEST WORKFLOW

A. Pull Request Creation

After completing a feature, the developer creates a Pull Request from the feature branch to main. The Pull Request should contain a clear description of the change, affected modules, and testing information.

GitHub describes Pull Requests as a mechanism for proposing, reviewing, and merging changes into a project.

B. Code Review

A second developer reviews the proposed changes. The reviewer checks:

- Correctness of implementation.
- Code readability.
- Possible bugs.
- Unnecessary changes.
- Security concerns.
- Compatibility with existing modules.
- Test coverage where applicable.

The reviewer can approve the Pull Request, add comments, or request changes.

C. Automated Testing

Where configured, automated checks can execute when a Pull Request is opened or updated. Typical checks include:

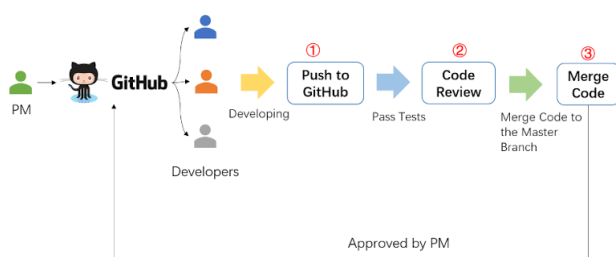
- Build verification.
- Unit tests.
- API tests.
- Code-quality checks.
- Security or dependency checks.

A failed check should be investigated before integration.

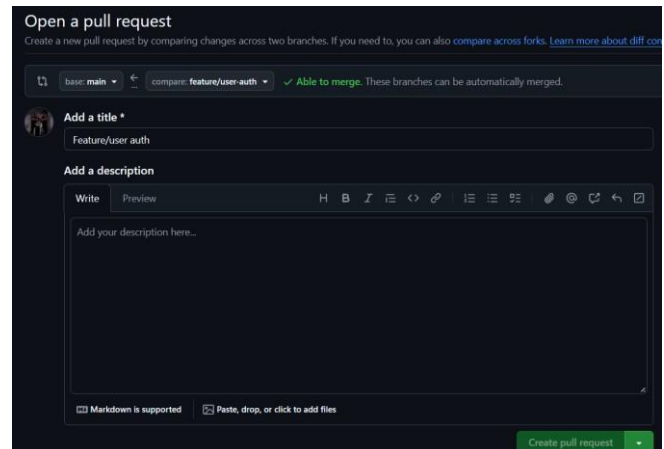
D. Approval and Merge

After required reviews and checks are completed, the Pull Request can be merged into main. GitHub supports multiple merge strategies; the selected method should follow the repository's defined policy.

Workflow



[FIGURE 3: Pull Request Workflow]



[SCREENSHOT 5: Pull Request]

IX. MERGE-CONFLICT RESOLUTION

A. Conflict Scenario

Consider two developers working on the Food Delivery Application.

Developer 3 works on feature/order-cart and modifies OrderService.js to update the order total. At approximately the same time, Developer 5 modifies the same section to add delivery charges.

For example, the original code contains:

```
const total = subtotal;
```

Developer 3 changes it to:

```
const total = subtotal - discount;
```

Developer 5 changes the same line to:

```
const total = subtotal + deliveryFee;
```

When one branch is merged into another, Git cannot automatically determine the intended final code.

B. Identification of Conflict

A conflict may occur when the developer executes:

```
git switch main
```

```
git pull origin main
```

```
git merge feature/order-cart
```

Git reports the conflicted file. The developer can identify the current state using:

```
git status
```

Git marks the conflicting section in the file.

C. Conflict Resolution Procedure

The conflicted file may contain markers similar to:

```
<<<<<<<< HEAD
```

```
const total = subtotal + deliveryFee;
```

```
=====
```

```
const total = subtotal - discount;
```

```
>>>>>>> feature/order-cart
```

CASE STUDY REPORT

The developer opens the file and determines the correct application logic. If both changes are required, the final code may be:

$\text{const total} = \text{subtotal} - \text{discount} + \text{deliveryFee};$

The conflict markers are removed and the file is saved.

The resolved file is then staged and committed:

`git add OrderService.js`

`git commit -m "Resolve order total merge conflict"`

`git push`

Git's documented conflict-resolution process is to edit the conflicted files, stage the resolved files with `git add`, and complete the merge with a commit or `git merge --continue`.

D. Verification After Resolution

After resolution, the developer should verify that:

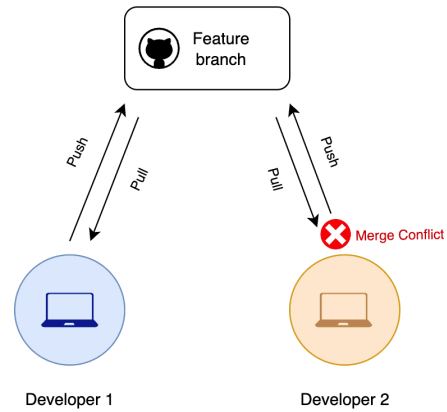
1. No conflict markers remain.
2. The application builds successfully.
3. Related tests pass.
4. The intended changes from both developers are retained.
5. The branch contains the correct final version.

If the merge should be cancelled instead of resolved, Git provides:

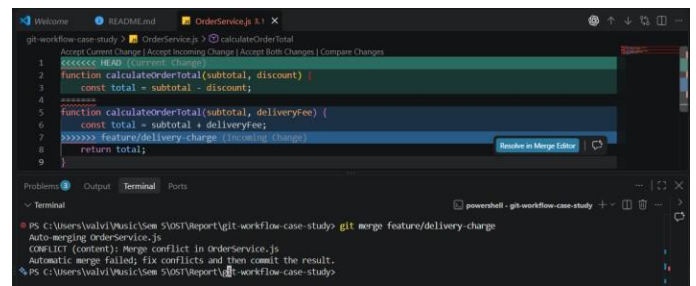
`git merge --abort`

TABLE V. MERGE-CONFLICT RESOLUTION STEPS

Step	Action
1	Run merge
2	Check git status
3	Open conflicted file
4	Identify conflict markers
5	Select or combine correct changes
6	Remove conflict markers
7	Run tests/build
8	git add resolved file
9	Commit the resolution
10	Push and verify



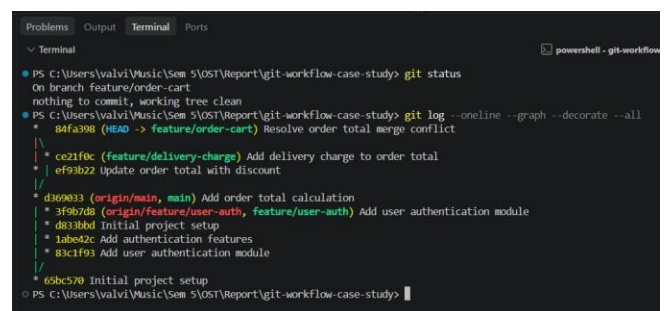
[FIGURE 4: Merge Conflict Resolution Process]



[SCREENSHOT 6: Merge Conflict]



[SCREENSHOT 7: Conflict Resolution]



[SCREENSHOT 8: Successful Merge]

X. SYSTEM ARCHITECTURE AND DESIGN

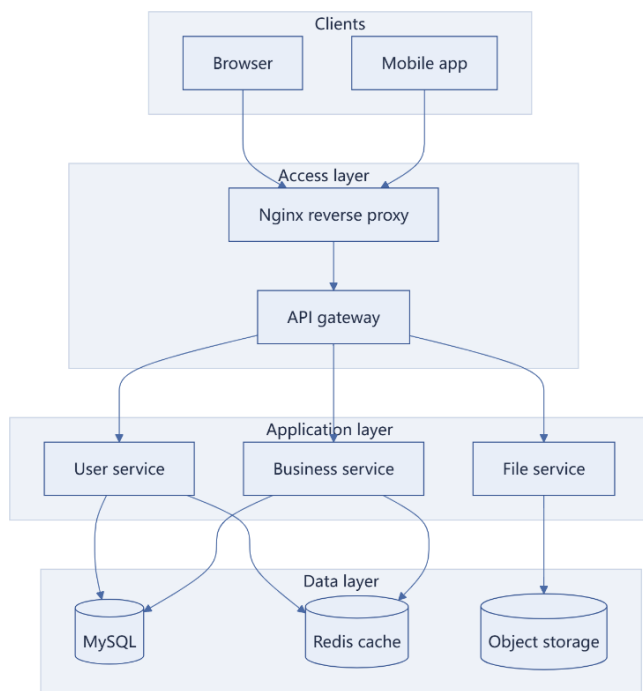
A. Proposed Architecture

The proposed system contains the Food Delivery Application as the main software system and Git/GitHub as the development-control layer.

The application can be organized into presentation, application/service, and data-access layers. Developers

CASE STUDY REPORT

independently modify application modules through feature branches.



[FIGURE 5: System Architecture]

B. Development Workflow Architecture

The development architecture follows:

```

Developer
↓
Local Repository
↓
Feature Branch
↓
Commit
↓
GitHub Repository
↓
Pull Request
↓
Code Review + Checks
↓
main Branch
  
```

This separates development from controlled integration.

C. Application Components

The representative application contains:

- User and authentication module.
- Restaurant and menu module.
- Cart and order module.
- Payment module.

- Delivery and tracking module.
- Administration module.

Each module can be developed independently where practical.

D. Database Structure

The application database may contain tables such as:

TABLE VI. REPRESENTATIVE DATABASE STRUCTURE

Table	Main Purpose
Users	Customer and user information
Restaurants	Restaurant details
Menu_Items	Food items and prices
Orders	Customer orders
Order_Items	Items included in each order
Payments	Payment information
Deliveries	Delivery and tracking information

This structure is illustrative and represents the application domain; it is not claimed as an implemented database.

XI. IMPLEMENTATION AND PROCEDURE

A. Initial Repository Setup

The project repository is created on GitHub and the initial source code is added to main.

A developer can clone the repository:

```

git clone <repository-url>
cd food-delivery-app
  
```

Repository access should be provided according to team responsibilities.

B. Branch Creation

Each developer updates main before starting work:

```

git switch main
git pull origin main
git switch -c feature/payment
  
```

This establishes the feature branch from the latest known project state.

C. Feature Development

The assigned developer implements the feature and records changes using meaningful commits.

```

git add .
git commit -m "Add payment validation"
  
```

CASE STUDY REPORT

Small logical commits make changes easier to understand during review.

D. Pull Request and Review

After development:

`git push -u origin feature/payment`

The developer creates a Pull Request and requests another team member to review the changes.

The reviewer checks the changed files and test results before approving or requesting modifications.

E. Conflict Resolution

If the feature branch conflicts with the latest main, the developer first synchronizes the branch and resolves the conflict locally.

A typical procedure is:

`git switch main`

`git pull origin main`

`git switch feature/payment`

`git merge main`

After resolving conflicts:

`git add .`

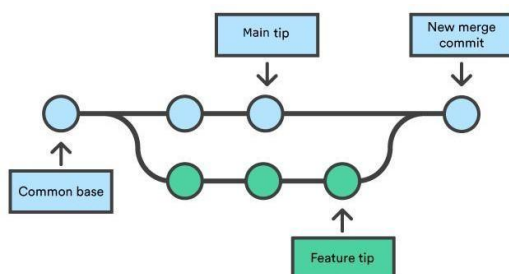
`git commit -m "Resolve merge conflicts"`

`git push`

The Pull Request is then updated and reviewed again.

F. Final Integration

After approval and successful checks, the Pull Request is merged into main. The feature branch may subsequently be deleted according to repository policy.



[FIGURE 6: Final Git Branch History]

XII. TESTING AND RESULTS

A. Testing Strategy

Testing should verify both the Git workflow and the application changes introduced through the workflow.

Testing is proposed at multiple levels:

- Git operation verification.
- Application build verification.
- Functional testing.
- Unit testing.
- Integration testing.
- Conflict-resolution verification.

The use of automated checks in a Pull Request workflow can help identify integration problems before changes are merged.

B. Test Cases

TABLE VII. PROPOSED TEST CASES

Test ID	Test Case	Expected Result
TC01	Clone repository	Repository is copied locally
TC02	Create feature branch	New branch is created
TC03	Commit feature change	Commit appears in history
TC04	Push branch	Branch appears on GitHub
TC05	Create Pull Request	PR is created successfully
TC06	Review changes	Reviewer can comment/approve
TC07	Protected main	Direct unauthorized push is blocked
TC08	Create merge conflict	Git identifies conflicting changes
TC09	Resolve conflict	Conflict markers are removed
TC10	Merge approved PR	Feature becomes part of main

C. Expected Results

The proposed workflow is expected to provide isolated feature development, controlled integration, traceable commits, and a defined procedure for resolving conflicts.

CASE STUDY REPORT

D. Obtained Results

No actual experimental results are claimed in this case study because implementation evidence, execution logs, screenshots, and measured test outputs were not provided. Therefore, the following classification applies:

- **Expected Result:** Defined by the proposed workflow and test cases.
- **Proposed/Illustrative Result:** Git should identify conflicting edits and allow their resolution through the documented process.
- **Actual Result:** Not available without execution evidence.

E. Testing Analysis

The proposed test cases cover the major activities required by the workflow. Actual implementation should additionally verify application-specific functionality and automated test results.

A successful Git workflow alone does not guarantee that the Food Delivery Application is functionally correct. Application testing remains necessary after integration.

XIII. SECURITY AND TECHNICAL ANALYSIS

A. Repository Security

The GitHub repository should use appropriate access permissions and protected branches. Direct modification of main should be restricted where possible. Sensitive configuration files should not be committed to the repository.

B. Access Control

Repository permissions should be assigned according to team responsibilities. Only authorized users should have write or administrative access. Pull Request review requirements can provide an additional control before changes enter the stable branch. [4]

C. Secret Management

Passwords, API keys, database credentials, and tokens should not be stored directly in source code. A .gitignore file can prevent common local configuration files from being committed. Secrets required by automated workflows should be stored using appropriate secret-management mechanisms rather than hard-coded values. If a secret is accidentally committed, simply deleting it from the latest file is not sufficient; the exposed credential should be revoked or rotated.

D. Reliability

Reliability is improved through regular synchronization, protected branches, review, testing, and a traceable commit history.

The team should avoid keeping feature branches isolated for unnecessarily long periods because larger differences can make later integration more difficult.

E. Maintainability

The workflow improves maintainability by providing:

- Consistent branch names.
- Meaningful commits.
- Review records.
- Clear ownership of features.
- Traceable changes.
- Controlled integration.

F. Scalability

The same workflow can support additional developers and modules if repository rules remain clear. For larger teams, additional controls such as CODEOWNERS, automated testing, issue tracking, and stricter branch rules can be introduced.

XIV. CHALLENGES AND SOLUTIONS

A. Merge Conflicts

Challenge: Multiple developers modify the same section of a file.

Solution: Developers should synchronize frequently, keep changes focused, and follow the defined conflict-resolution procedure.

B. Overwritten Changes

Challenge: Direct modification of shared files or branches can cause one developer's changes to replace another's work.

Solution: Use feature branches and Pull Requests instead of direct development on main.

C. Code Review Issues

Challenge: Reviewers may miss defects or developers may submit excessively large Pull Requests.

Solution: Keep Pull Requests focused, provide clear descriptions, and require review before merging.

D. Branch Management

Challenge: Unclear branch names and unused branches make repository management difficult.

Solution: Follow a fixed naming convention and remove obsolete feature branches after successful integration where appropriate.

CASE STUDY REPORT

E. Proposed Solutions

TABLE VIII. CHALLENGES AND SOLUTIONS

Challenge	Proposed Solution
Code overwriting	Feature branches
Merge conflicts	Regular synchronization and conflict resolution
Unreviewed code	Pull Request requirement
Unstable main branch	Branch protection
Poor traceability	Meaningful commits
Incorrect integration	Automated checks
Unclear ownership	Defined responsibilities
Exposed credentials	Secret management

XV. SDG MAPPING

A. SDG 9 – Industry, Innovation and Infrastructure

SDG 9 focuses on industry, innovation, and infrastructure. A structured Git-based development workflow supports software engineering infrastructure by enabling organized collaboration, version management, and reliable integration. The proposed workflow can help development teams establish repeatable engineering practices for building and maintaining software systems.

B. SDG 17 – Partnerships for the Goals

SDG 17 emphasizes partnerships and collaboration. GitHub-based development supports collaboration by allowing multiple developers to work on a common software project while maintaining separate branches and reviewing changes before integration.

Pull Requests provide a documented mechanism for discussion, review, and shared decision-making within the development team.

C. Contribution of the Proposed Workflow

TABLE IX. SDG MAPPING

SDG	Connection	Contribution
SDG 9	Software engineering infrastructure	Structured version control and integration
SDG 17	Collaborative development	Shared repository, review, and communication

The mapping represents the potential contribution of the proposed workflow and does not claim a measured SDG impact.

XVI. RECOMMENDATIONS

A. Development Practices

Developers should:

- Work through feature branches.
- Keep commits focused.
- Use meaningful commit messages.
- Pull the latest main before starting work.
- Synchronize regularly.
- Avoid unnecessary changes to unrelated files.

B. Repository Management

The repository should use:

- Protected main branch.
- Standard branch names.
- Pull Request requirements.
- Review rules.
- Clear repository permissions.
- Periodic cleanup of obsolete branches.

C. Testing and Security

Automated tests should be integrated into the Pull Request process where practical. Developers should also avoid committing credentials and should review dependencies and configuration files for security risks.

D. Future Improvements

The workflow can later be extended with:

- GitHub Actions for automated testing.
- CODEOWNERS for module-specific reviews.
- Automated code-quality checks.
- Dependency and security scanning.
- Issue-based development.
- Release branches when project size requires them.

These improvements should be introduced according to project requirements rather than adding unnecessary process overhead.

CASE STUDY REPORT

XVII. CONCLUSION

This case study examined a collaborative software development problem involving five developers working on a Food Delivery Application. The existing uncontrolled workflow can lead to overwritten changes, inconsistent versions, merge conflicts, and delays in integration. A Git and GitHub-based workflow was proposed using a protected main branch and separate feature branches. Developers work independently, commit their changes, push feature branches to GitHub, and submit Pull Requests for review before integration. This provides a controlled path from development to the stable codebase. The merge-conflict example demonstrates that Git can identify competing modifications, but the final resolution requires developer understanding of the intended application behavior. The proposed testing and security practices further support reliable repository management. The case study does not claim measured implementation results because no execution evidence was provided. Instead, it defines an illustrative workflow and expected outcomes that can be validated through actual implementation. Overall, the proposed Git-based workflow provides a practical structure for collaborative development by

separating feature work from stable integration, improving change traceability, and establishing a consistent process for code review and conflict resolution.

REFERENCES

- [1] S. Chacon and B. Straub, *Pro Git*, 2nd ed. Berkeley, CA, USA: Apress, 2014.
- [2] GitHub, "Pull Requests Documentation," GitHub Docs, 2026.
- [3] M. Fowler, "Continuous Integration," Martin Fowler, updated Jan. 18, 2024.
- [4] GitHub, "Managing Protected Branches," GitHub Docs, 2026.
- [5] GitHub, "About Pull Requests," GitHub Docs, 2026.
- [6] GitHub, "Pull Request Reviews," GitHub Docs, 2026.
- [7] Git SCM, "git-merge Documentation," Git Documentation, 2026.
- [8] J. Loeliger and M. McCullough, *Version Control with Git*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2012.